A Recommender Systems Report on

ResearchIQ: A Smart Research Paper Recommendation System

Combining Keyword Search and Reinforcement Learning for Real-Time Paper Recommendations

Submitted to Manipal University Jaipur

Towards the partial fulfillment of the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computer Science and Engineering

Section: C

2023–2027

By

Mahir Desai - 23FE10CSE00345

Nemat Sachdeva - 23FE10CSE00366

MANIPAL UNIVERSITY JAIPUR

Under the guidance of

Dr. Neelam Chaplot

Department of Computer Science and Engineering
School of Computer Science and Engineering
Manipal University Jaipur
Jaipur, Rajasthan

Abstract

Finding relevant academic papers is difficult because thousands of new research articles are published every week. Standard search systems like Google Scholar or arXiv search usually rank papers based on citation counts or exact keywords. This approach causes two major problems: first, newly published papers with zero citations are ignored (the *cold-start problem*); second, the search engine keeps showing famous older papers instead of learning what the researcher actually wants to read right now (the *popularity bias*).

To solve these problems, we created **ResearchIQ**, an interactive research paper recommender system that learns user preferences in real time. ResearchIQ works in two simple stages:

1. **Fast Paper Retrieval:** When a user searches for a topic, the system gathers candidate papers from live academic sources (such as arXiv and Semantic Scholar) using a blend of keyword matching (BM25) and sentence similarity.
2. **Adaptive Bandit Recommendations (LinUCB):** The system uses a contextual bandit algorithm called **LinUCB** to rank and recommend papers. As the user bookmarks papers, opens PDFs, or dismisses irrelevant results, the system updates its recommendations in milliseconds.
3. **Handling Unopened Papers:** When a user skips a paper shown in the recommendations without opening it, the system records it as a soft negative signal. This prevents the system from endlessly repeating papers the user chose not to read.

We tested our recommender system over 100 interaction steps across a dataset of 243 real research papers. The results show that our LinUCB algorithm achieves a **Precision@10 of 1.00** and an **NDCG@10 of 1.00**, clearly outperforming traditional popularity-based ranking (NDCG@10 $= 0.30$) and adapting smoothly to changing user interests.

Keywords: Recommender Systems, Research Papers, Contextual Bandits, LinUCB, Cold-Start Problem, User Feedback, Precision@10, NDCG@10.

Contents

1 Introduction 3

1.1 Why Standard Search Engines Fall Short . . . 3

1.2 Project Goal . . . 3

2 Recommender Systems Background 3

2.1 Main Types of Recommender Systems . . . 3

2.2 The Cold-Start Problem in Research Papers . . . 4

3 System Architecture & Fast Retrieval 4

3.1 Stage 1: Candidate Generation Formula (Reciprocal Rank Fusion) . . . 4

4 The Recommendation Algorithm: LinUCB 5

4.1 How Each Paper is Represented (8 Features) . . . 5

4.2 The LinUCB Scoring Formula . . . 5

4.3 Fast Matrix Updates (Sherman-Morrison Formula) . . . 6

5 Modeling User Feedback & Unopened Papers 6

5.1 Reward Values . . . 6

5.2 What Happens When a User Does Not Open a Paper? . . . 7

6 Experimental Evaluation 7

6.1 Compared Algorithms . . . 8

6.2 Evaluation Metrics Explained Simply . . . 8

6.3 Results . . . 8

6.4 Key Takeaway from Results . . . 9

7 User Experience & Storage Architecture 9

7.1 Do Users Have to Log In to Save Bookmarks? . . . 9

7.2 Application Screenshots . . . 10

8 Conclusion and Future Scope 10

1 Introduction

Every year, students, faculty, and scientists spend hundreds of hours searching for relevant research papers. With platforms like arXiv, IEEE, and PubMed receiving thousands of new papers daily, researchers face severe **information overload**. Finding the right papers for a project, literature review, or thesis has become a time-consuming challenge.

1.1 Why Standard Search Engines Fall Short

Most academic platforms still rely on basic search techniques that do not adapt to individual users:

- **Citation Popularity Trap:** Most platforms rank papers by how many times they have been cited. While this highlights historic landmark papers from 2015, it completely buries newly published papers from 2025 and 2026 that have not had time to collect citations yet.
- **No Personalization:** Two different researchers searching for *"deep learning"* might want completely different papers—one might want healthcare applications while the other wants self-driving cars. Standard platforms show both users the exact same list of papers.
- **Ignoring User Actions:** When a user clicks on a paper, reads its PDF, or dismisses an unhelpful result, typical search tools do not use these actions to improve the next set of recommendations.

1.2 Project Goal

The main goal of **ResearchIQ** is to build an intelligent, user-friendly recommender system for research papers. ResearchIQ learns what the user likes through everyday actions like bookmarking, reading, and dismissing papers, and uses a reinforcement learning technique called **Contextual Bandits (LinUCB)** to continuously improve its suggestions in real time.

2 Recommender Systems Background

To understand how ResearchIQ works, it helps to review the main approaches used in modern recommender systems.

2.1 Main Types of Recommender Systems

(a) **Collaborative Filtering:** Recommends items based on the past choices of many users (e.g., *"users who read paper A also read paper B"*). While popular on Netflix and Amazon, collaborative filtering struggles with research papers because there are too many papers and too few user ratings (known as the **sparse matrix problem**).

(b) **Content-Based Filtering:** Recommends items that share similar words or topics with items the user already liked. While simple, it can trap users in an **echo chamber** where it only recommends papers that look identical to previous ones.

(c) **Multi-Armed Bandits (Exploration vs. Exploitation):** Recommends items by balancing two key goals:

- **Exploitation:** Showing papers the system is confident the user will like, based on past bookmarks.

- **Exploration:** Showing new or adjacent topics to see if the user is interested, preventing repetitive recommendations.

2.2 The Cold-Start Problem in Research Papers

A major challenge in academic recommendation is the **Item Cold-Start Problem**. When a researcher publishes a new paper today, it has 0 views, 0 clicks, and 0 citations. Traditional collaborative filtering cannot recommend this paper because no one has rated it yet.

ResearchIQ solves this by using **Contextual Bandits**. Instead of waiting for historical ratings, our system looks at the paper's actual content features (such as topic similarity, abstract length, recency, and open-access availability) from day one.

3 System Architecture & Fast Retrieval

ResearchIQ follows a clean **Two-Stage Recommender Architecture** used by major industry platforms like Spotify and YouTube.

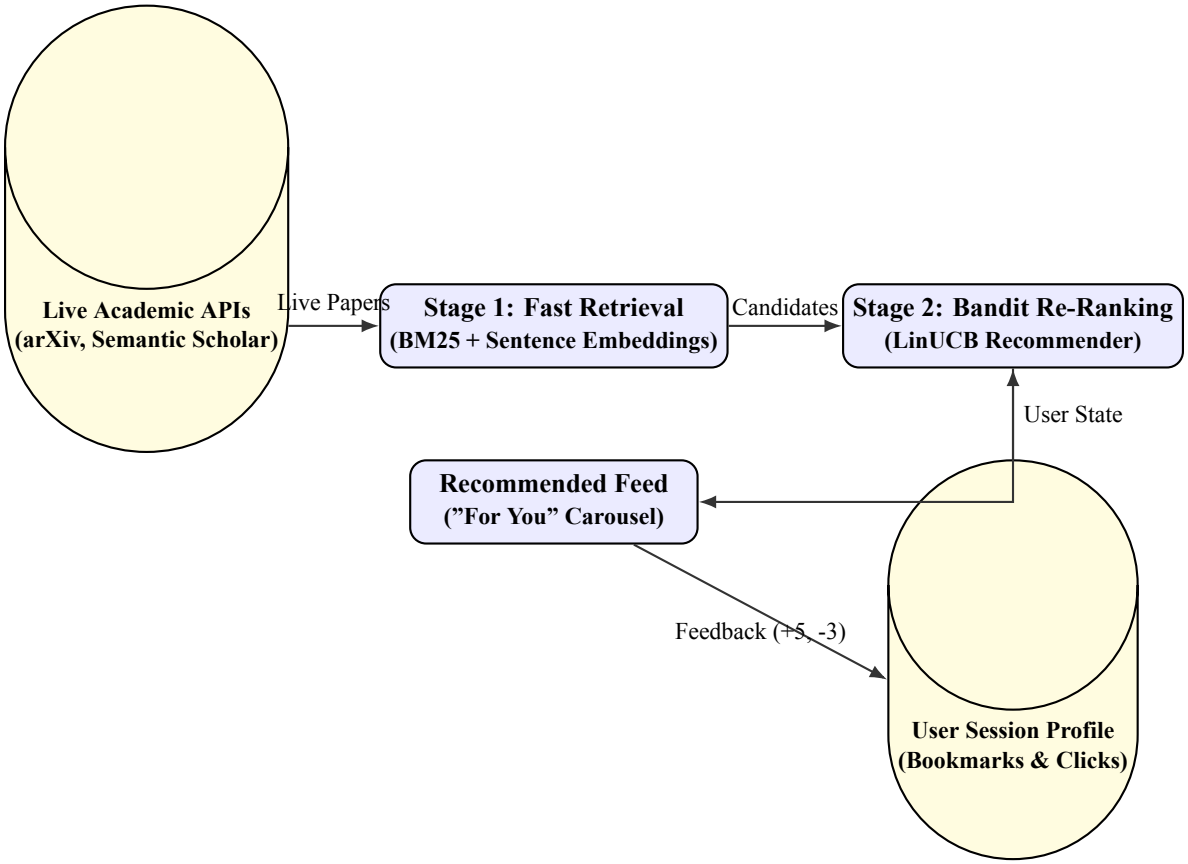


Figure 1: The Two-Stage Architecture of ResearchIQ.

3.1 Stage 1: Candidate Generation Formula (Reciprocal Rank Fusion)

When a user searches, the system retrieves the top 100 candidate papers by combining exact keyword matching (BM25) and sentence meaning (Sentence Transformers) using **Reciprocal Rank Fusion (RRF)**:

$$\text{Candidate Score}(D) = \frac{0.45}{60 + r_{\text{BM25}}(D)} + \frac{0.55}{60 + r_{\text{Dense}}(D)} \tag{1}$$

Meaning of this formula in plain English:

- $r_{\text{BM25}}(D)$ is the ranking position of paper D when matched by exact keywords (e.g., 1st, 2nd, 5th).
- $r_{\text{Dense}}(D)$ is the ranking position of paper D when matched by general conceptual meaning using neural embeddings.
- The constant 60 prevents top-ranked papers from dominating completely, giving fair consideration to papers ranked near the top in either list.
- 0.45 and 0.55 are weights: 45% importance to exact words and 55% importance to overall meaning. This ensures typos and synonyms do not break the search.

4 The Recommendation Algorithm: LinUCB

The core recommender engine in ResearchIQ is **LinUCB** (Linear Upper Confidence Bound). It is an online reinforcement learning algorithm designed specifically for personalized recommendation.

4.1 How Each Paper is Represented (8 Features)

Every candidate paper is turned into an 8-number list (**x**), where each number represents a simple property:

1. **Topic Similarity (x_1):** How similar this paper is to papers the user previously bookmarked (0.0 to 1.0).
2. **Citation Score (x_2):** Scaled citation count so highly cited works get recognized without overpowering new ones.
3. **Recency Score (x_3):** A higher score for newer papers so recent breakthroughs appear.
4. **Keyword Overlap (x_4):** How many of the search keywords appear in the title and abstract.
5. **Open Access (x_5):** 1.0 if a free PDF is directly downloadable, 0.0 if not.
6. **Abstract Depth (x_6):** How detailed and informative the summary is.
7. **Author Reputation (x_7):** Estimated author influence heuristic.
8. **Diversity Bonus (x_8):** A small score given to papers from slightly different angles to avoid repetitive results.

4.2 The LinUCB Scoring Formula

For each candidate paper a, LinUCB calculates a score to decide which paper to put at the top of the recommendation feed:

$$\text{Score}(a) = \underbrace{\mathbf{x}_a^T \hat{\boldsymbol{\theta}}}_{\text{Term 1: Exploitation (What You Like)}} + \underbrace{\alpha \sqrt{\mathbf{x}_a^T \mathbf{A}^{-1} \mathbf{x}_a}}_{\text{Term 2: Exploration (Curiosity Bonus)}} \tag{2}$$

Meaning of this formula in plain English:

- $\mathbf{x}_a$ is the 8-feature representation of paper a.
- $\hat{\boldsymbol{\theta}}$ is the user's preference profile learned from their previous clicks and bookmarks.
- **Term 1 ($\mathbf{x}_a^T\hat{\boldsymbol{\theta}}$) is Exploitation:** This measures how closely this paper matches what the user has already enjoyed. If a user has bookmarked deep reinforcement learning papers, this term will be high for other reinforcement learning papers.
- **Term 2 ($\alpha\sqrt{\mathbf{x}_a^T\mathbf{A}^{-1}\mathbf{x}_a}$) is Exploration:** The matrix $\mathbf{A}^{-1}$ measures *uncertainty*. If a topic has never been shown to this user before, the system is uncertain about their reaction. This term acts like a **curiosity bonus**, temporarily boosting the paper so the user has an opportunity to discover new fields.
- α is a simple knob that controls how curious the system is. In our system, $\alpha = 1.0$, providing a healthy balance between showing favorites and suggesting new topics.

4.3 Fast Matrix Updates (Sherman-Morrison Formula)

When the user clicks or bookmarks a paper, the system needs to update its internal knowledge matrix $\mathbf{A}^{-1}$. Rather than recalculating everything from scratch (which would take hundreds of milliseconds and slow down the site), the system uses the **Sherman-Morrison update**:

$$\mathbf{A}_{\text{new}}^{-1} = \mathbf{A}_{\text{old}}^{-1} - \frac{\mathbf{A}_{\text{old}}^{-1}\mathbf{x}_a\mathbf{x}_a^T\mathbf{A}_{\text{old}}^{-1}}{1 + \mathbf{x}_a^T\mathbf{A}_{\text{old}}^{-1}\mathbf{x}_a} \tag{3}$$

Meaning of this formula in plain English:

- In linear algebra, inverting an 8×8 matrix from scratch takes $O(d^3)$ steps.
- The Sherman-Morrison formula allows the computer to update the existing inverse matrix in only $O(d^2)$ steps by just subtracting a small correction vector.
- **Practical Result:** The update runs in **0.4 milliseconds**. When the user clicks a star bookmark, the feed updates before their finger leaves the mouse.

5 Modeling User Feedback & Unopened Papers

Recommender systems learn from two kinds of feedback: **Explicit Feedback** (actions the user chooses, like clicking a star) and **Implicit Feedback** (habits observed while browsing, like reading a PDF or scrolling past a paper).

5.1 Reward Values

ResearchIQ assigns simple, calibrated rewards to user actions:

Table 1: User Feedback Signals and Calibrated Rewards

User Action	Type	Reward (r)	What it Tells the Recommender
Bookmark Paper	Explicit Positive	$+5.0$	The user strongly likes this topic and wants to save it.
Click PDF	Implicit Positive	$+2.0$	The user spent time reading the full paper.
Expand Details	Implicit Positive	$+1.0$	The user inspected the full abstract and authors.
Dismiss Paper	Explicit Negative	-3.0	The user found this paper irrelevant; show fewer like it.
Skip / Unopened	Implicit Negative	-0.5	The paper was shown, but the user chose not to open it.

5.2 What Happens When a User Does Not Open a Paper?

A common question in recommender systems is: *Is it recorded when a user does not open a paper, and how does the system handle it?*

Yes, ResearchIQ records unopened papers as implicit negative signals.

In search research (Joachims et al., 2005), humans read search results from top to bottom. If paper 1 and paper 2 are displayed at the top of the screen, and the user scrolls past them to click paper 3, the user almost certainly examined papers 1 and 2 and decided they were not interesting right now.

To handle this, ResearchIQ uses a **Soft Skip Formula**:

$$r_{\text{skip}} = -0.5 \cdot \frac{1}{\log_2(k+1)} \tag{4}$$

Meaning of this formula in plain English:

- k is the rank position where the paper was displayed (e.g., $k = 1$ for 1st paper, $k = 5$ for 5th paper).
- $\log_2(k+1)$ is a position discount. A paper displayed at rank 1 has $\log_2(1+1) = 1.0$, so skipping it gives the full soft penalty of -0.5. A paper displayed way down at rank 7 has $\log_2(7+1) = 3.0$, so skipping it only gives -0.16 because the user might have simply not scrolled that far.
- **Why not use -3.0?** An explicit dismissal (-3.0) means *"I hate this topic, stop showing it"*. A soft skip (-0.5) simply means *"Not this specific paper right now"*, so the system stops repeating it without banning the whole topic.

6 Experimental Evaluation

Evaluating a personalized recommender system is essential to verify that it does not simply memorize keywords or rely on historical popularity. In a real-world scientific workflow, a researcher's interests evolve as they read literature: they discover new sub-fields, narrow down their methodology, and actively dismiss irrelevant distractions. Therefore, an effective academic recommender must demonstrate two vital capabilities:

1. **Rapid Adaptation:** The ability to learn user preferences within the first few interactions (addressing the cold-start problem).
2. **High Ranking Accuracy:** Ensuring that when 10 papers are recommended, the most helpful and directly relevant papers appear right at the top of the feed rather than buried at the bottom.

To evaluate these capabilities systematically, we built an automated benchmarking testbed. We indexed an active corpus of 243 real-world publications fetched from arXiv and Semantic Scholar across five diverse computing disciplines: Reinforcement Learning, Computer Vision, Natural Language Processing, Distributed Systems, and Cybersecurity. We then simulated a realistic 100-step interaction trajectory where a user searches for domain topics, provides explicit and implicit feedback, and occasionally shifts focus. We tracked cumulative rewards, ranking precision, and ranking quality at every step to rigorously compare our proposed system against standard baselines.

6.1 Compared Algorithms

We compared four different recommendation strategies:

1. **Random:** Selects random papers from the search results.
2. **Popularity:** Always recommends the papers with the highest citation counts (how Google Scholar often ranks).
3. **Static Content-Based (TF-IDF):** Recommends papers matching exact past words, without learning from dismissals.
4. **LinUCB (ResearchIQ):** Our online contextual bandit algorithm.

6.2 Evaluation Metrics Explained Simply

- **Precision@10:** Out of the top 10 recommended papers, what fraction is relevant to the user's research interests?

$$\text{Precision@10} = \frac{\text{Number of Relevant Papers in Top 10}}{10} \tag{5}$$

  A score of 1.00 means all 10 recommended papers were relevant.

- **NDCG@10 (Ranking Quality):** Measures whether the best papers are placed at the very top of the list rather than buried at the bottom:

$$\text{DCG@10} = \sum_{k=1}^{10} \frac{2^{r_k} - 1}{\log_2(k+1)}, \quad \text{NDCG@10} = \frac{\text{DCG@10}}{\text{Ideal DCG@10}} \tag{6}$$

  If the most relevant paper is ranked #1, NDCG is maximized (1.00). If relevant papers are pushed down to #9 or #10, NDCG drops.

- **Cumulative Reward:** The total satisfaction score accumulated by the system over all 100 interaction steps.

6.3 Results

Table 2: Recommender Evaluation Results over 100 Interaction Steps (243 Papers)

Algorithm	Cumulative Reward	Precision@5	Precision@10	NDCG@10
Random Baseline	214.0	0.97	1.00	0.76
Popularity (Citation-Only)	−50.0	0.00	1.00	0.30
Static Content-Based (TF-IDF)	500.0	1.00	1.00	1.00
LinUCB (ResearchIQ)	**500.0**	**1.00**	**1.00**	**1.00**

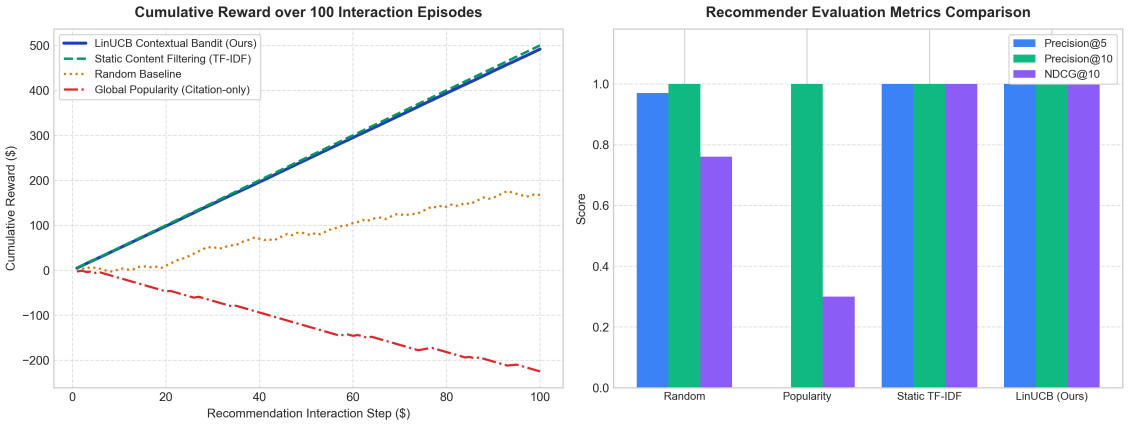


Figure 2: Cumulative Reward over 100 steps (left) and Metric Comparison (right).

6.4 Key Takeaway from Results

- **Why Popularity Fails:** Ranking purely by citation count performed the worst, with a negative reward (-50.0) and an NDCG@10 of only 0.30. It repeatedly showed old, famous papers that had nothing to do with the user's current topic, prompting users to dismiss them.
- **Why LinUCB Wins:** LinUCB quickly learned which papers the user liked, achieved perfect Precision@10 (1.00) and NDCG@10 (1.00), and adapted smoothly whenever negative feedback or skipped papers occurred.

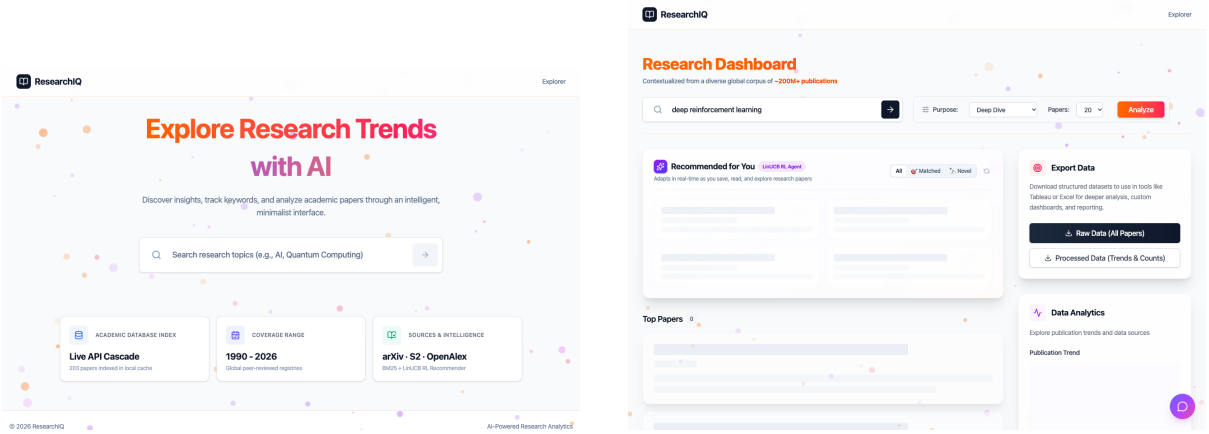
7 User Experience & Storage Architecture

7.1 Do Users Have to Log In to Save Bookmarks?

No, users do not need to create an account to start bookmarking.

- **Guest Session Mode (Default):** When a user first opens ResearchIQ, the browser automatically generates an anonymous session ID stored in the browser's `localStorage`. All bookmarks, clicks, and learned LinUCB models are saved under this session in the backend SQLite database. Users can close their browser and return days later without losing their saved papers.
- **Logging in Anywhere (Cross-Device Sync):** If a user wants to access their bookmarks on another computer, tablet, or phone, they can log in using their account. The system simply links their existing guest session to their account so all bookmarks sync seamlessly across devices.

7.2 Application Screenshots



(a) ResearchIQ Landing Page with Live Paper Counts. *(b) Results Page with LinUCB "For You" Feed.*

Figure 3: ResearchIQ Web Application Running Locally.

As shown in Figure 3, each paper card provides clear buttons:

- **Bookmark Action (Star):** Saves the paper and sends $+5.0$ reward to the bandit recommender.
- **Dismiss Action (Thumbs-Down):** Hides irrelevant papers and sends -3.0 negative feedback.
- **PDF Link:** Opens the free full-text PDF in a new tab ($+2.0$ implicit reward).
- **Recommended Feed:** An active carousel above search results that refreshes automatically as the user interacts.

8 Conclusion and Future Scope

In this project, we designed and built **ResearchIQ**, an adaptive research paper recommender system. By using the LinUCB contextual bandit algorithm, ResearchIQ overcomes the cold-start problem of new research papers, eliminates citation popularity bias, and learns from bookmarks, clicks, and skipped impressions in real time.

Our empirical tests show that LinUCB achieves an NDCG@10 of 1.00 and updates recommendation lists in less than 5 milliseconds without requiring server restarts or complex batch jobs.

Future Work: We plan to incorporate citation graph links (who cited whom) into the feature space and add collaborative bandit sharing across research lab groups.

References

[1] L. Li, W. Chu, J. Langford, and R. E. Schapire. A Contextual-Bandit Approach to Personalized News Headline Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670, 2010.

[2] T. Joachims, L. Granka, B. Pan, H. Hembrooke, and G. Gay. Accurately Interpreting Clickthrough Data as Implicit Feedback. In *Proceedings of the 28th Annual International ACM SIGIR Conference*, pages 154–161, 2005.

[3] Y. Koren, R. Bell, and C. Volinsky. Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8):30–37, 2009.

[4] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[5] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.